

Adam and AdamW

How far to move each weight · video 2:46

This lesson answers one question: once backprop has given every weight a gradient, how far should each weight move? Plain gradient descent moves it by learning rate $\times$ gradient, so its steps follow the gradient's scale. **Adam** normalizes each weight's step by the typical size of its gradients, using running averages m and v . In a tiny GPT it beats SGD and momentum. **AdamW** adds weight decay separately from that normalization; putting the decay into the gradient instead (Adam + L2) collapses the weights at the same coefficient.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Why plain gradient descent struggles	▶ 0:08-0:41	2	☐
2. Adam's idea	▶ 0:41-1:11	2	☐
3. Training the tiny GPT	▶ 1:11-1:29	1	☐
4. Weight decay: Adam + L2 vs AdamW	▶ 1:29-2:20	2	☐
5. The cost, and the code	▶ 2:20-2:38	2	☐

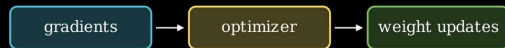
Before you start: Deep Dive episode 18 (backprop) and Foundations (gradient descent). Code: `code/d19_adam` (PyTorch only; eight tiny models, about 25 minutes on a CPU).

CONCEPT 1 OF 5

Why plain gradient descent struggles

▶ Video 0:08-0:41

Backprop gives every weight a gradient; the **optimizer** decides how far to move each one. Plain gradient descent (SGD) uses $w \leftarrow w - lr \cdot g$, so the step depends on the scale of the gradient. Multiply the loss by 1,000 and SGD's first step (largest change in any weight, lr 0.01) grows from **0.0244** to **24.4**. Adam's first step is **0.0100** both times.



almost every LLM: AdamW

gradient descent: $w \leftarrow w - lr \cdot g$

first step size	loss × 1	loss × 1,000
SGD	0.0244	24.3951
Adam	0.0100	0.0100

SGD's step follows the gradient's scale; Adam's does not

▶ 0:19 Gradients → optimizer → weight updates.
Almost every LLM uses AdamW.

▶ 0:39 Loss × 1,000: SGD's first step grows from 0.0244 to 24.4; Adam's stays 0.0100.

KEY IDEA

SGD's step size depends on the gradient's scale; Adam's is set by the learning rate.

Check 1: Why plain gradient descent struggles

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

With SGD and lr 0.01, a weight's gradient is 50. How far does it move?

1.2 TRUE OR FALSE

"Multiplying the loss by 1,000 multiplies SGD's update by 1,000."

True ☐ False ☐

CONCEPT 2 OF 5

Adam's idea

▶ Video 0:41-1:11

Adam keeps two running averages for every weight: **m**, the average gradient ($m \leftarrow 0.9 m + 0.1 g$), and **v**, the average squared gradient ($v \leftarrow 0.999 v + 0.001 g^2$). The step is $\text{lr} \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$: the direction comes from m, the size is normalized by how big that weight's gradients usually are. Both averages start at zero, so early on they are divided by $(1 - 0.9^t)$ and $(1 - 0.999^t)$, the **bias correction**.

```
m ← 0.9 · m + 0.1 · g      average gradient
v ← 0.999 · v + 0.001 · g2 average squared gradient
w ← w - lr · m̂ / (√v̂ + ε)

direction from m; size normalized by that weight's usual gradient
m̂, v̂: corrected upward early on (both start at zero)
```

```
our Adam vs torch.optim.Adam, 10 steps:
largest difference 3.0 × 10-8
```

▶ 1:02 m: running average of the gradient; v: of its square; step = $\text{lr} \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$.

▶ 1:09 Our 10-line Adam matches torch.optim.Adam after 10 steps to 3×10^{-8} .

Our own Adam, about ten lines, matches PyTorch's after ten steps to within 3×10^{-8} .

KEY IDEA

Adam = momentum (m) + a per-weight step size ($1/\sqrt{v}$): every weight moves about lr per step at first, whatever the scale of its gradient.

Check 2: Adam's idea

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 SHORT ANSWER

At step 1, $\hat{m} = g$ and $\hat{v} = g^2$. What is the step for every weight, and why is the first step 0.0100 in the episode?

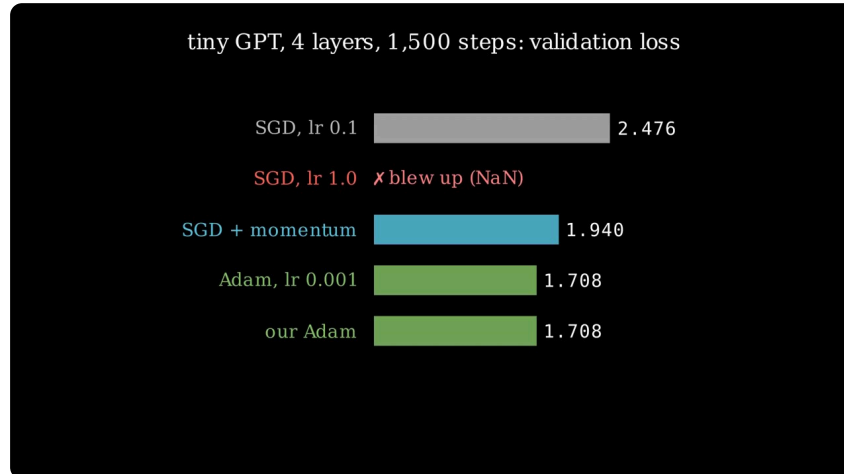
2.2 CALCULATION

At step 1, by what factors do the bias corrections scale m and v?

Training the tiny GPT

▶ Video 1:11-1:29

The same 4-layer tiny GPT, 1,500 steps. Validation loss: SGD with lr 0.1 **2.476**; with lr 1.0 it blows up (NaN); SGD with momentum 0.9 and lr 0.1 **1.940**; Adam with lr 0.001 **1.708**; our hand-written Adam exactly the same, 1.708.



▶ 1:28 1,500 steps: SGD lr 0.1 2.476; lr 1.0 blew up; momentum 1.940; Adam 1.708; our Adam 1.708.

KEY IDEA

On transformers, Adam trains faster and more reliably than SGD, with less tuning of the learning rate.

Check 3: Training the tiny GPT

Answer all 1 questions before looking at the answers. Pass mark: all correct.

3.1 PUT IN ORDER

Order from best to worst validation loss: *SGD lr 0.1* · *Adam* · *SGD + momentum*.

.....

CONCEPT 4 OF 5

Weight decay: Adam + L2 vs AdamW

▶ Video 1:29–2:20

Weight decay pulls weights toward zero to keep them small. **Adam + L2** adds $wd \cdot w$ to the gradient; **AdamW** shrinks each weight directly, $w \leftarrow w - lr \cdot wd \cdot w$, separately from the gradient step.

With Adam the first way backfires: the decay term is normalized like any gradient, so a weight whose loss gradient is small gets pushed toward zero by about lr each step, whatever its size. With wd 0.1 and lr 0.003, the total size of the weights collapses from **173.8** (no decay) to **4.0**, and the loss rises to **3.307**. AdamW: size **127.0**, loss **1.682** (no decay: 1.666).

weight decay: pull weights toward zero

Adam + L2

$$g \leftarrow g + 0.1 \cdot w$$

(then normalized like any gradient)

AdamW

$$w \leftarrow w - lr \cdot 0.1 \cdot w$$

(a small direct shrink, every step)

▶ 1:43 Adam + L2 adds $0.1 \cdot w$ to the gradient; AdamW shrinks the weight directly by $lr \cdot 0.1 \cdot w$.

weight decay 0.1, lr 0.003, 1,500 steps

Adam + L2: every weight pushed toward zero by $\sim lr$ per step, whatever its size

total size of the weights

no decay	173.8
Adam + L2	4.0
AdamW	127.0

val loss: no decay 1.666 · Adam + L2 3.307 · AdamW 1.682

▶ 2:05 Weight size: no decay 173.8, Adam + L2 4.0, AdamW 127.0; val loss 1.666, 3.307, 1.682.

To be fair, the same coefficient means very different things in the two methods; Adam + L2 would need a much smaller one. AdamW keeps decay separate from Adam's normalization, so it behaves predictably: that is why it became the default.

KEY IDEA

In AdamW, weight decay is a plain shrink by $(1 - lr \cdot wd)$ per step, independent of the gradients.

Check 4: Weight decay: Adam + L2 vs AdamW

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

With AdamW, lr 0.003 and wd 0.1, by what factor would decay alone shrink a weight over 1,500 steps?

4.2 MULTIPLE CHOICE

Why did Adam + L2 collapse the weights?

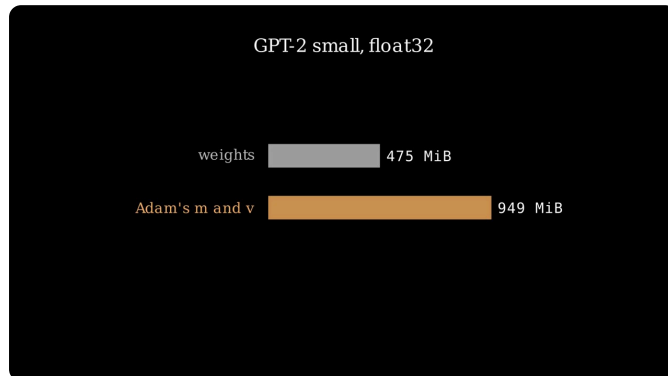
- A. The learning rate was too small
- B. The decay term was normalized by $\sqrt{v}$, so it moved every weight by about lr per step
- C. AdamW has a bug
- D. Weight decay was turned off

CONCEPT 5 OF 5

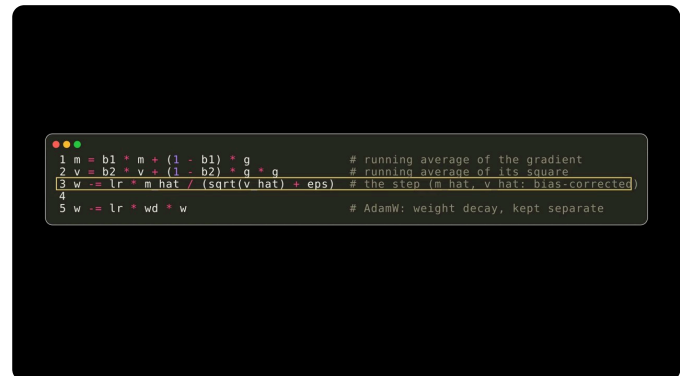
The cost, and the code

▶ Video 2:20–2:38

The price: two extra numbers per weight. For GPT-2 small in float32, **475 MiB** of weights need **949 MiB** more for Adam's m and v. In code, the heart of Adam is three lines: update m, update v, and step by $\hat{m} / (\sqrt{\hat{v}} + \epsilon)$; AdamW adds one separate line for the decay.



▶ 2:29 GPT-2 small: 475 MiB of weights, plus 949 MiB for Adam's m and v.



▶ 2:37 Three lines: update m, update v, step by m over $\sqrt{v}$; AdamW decays separately.

KEY IDEA

Adam triples the memory needed for parameters (weights + m + v), before counting gradients and activations.

Check 5: The cost, and the code

Answer all 2 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

How many GB do Adam's m and v take for a 7-billion-parameter model in float32?

5.2 HANDS-ON CODE

Try it yourself. Repeat part 2 with lr 0.05. What is Adam's first step for each weight?

```
opt = torch.optim.Adam([w], lr=0.05)
((x @ w) ** 2).mean().backward(); opt.step()
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Why plain gradient descent struggles · rewatch ▶ 0:08-0:41

1.1 0.5. $\text{lr} \times g = 0.01 \times 50$.

1.2 True. The gradient scales by 1,000 and SGD's update is proportional to it ($0.0244 \rightarrow 24.4$).

2. Adam's idea · rewatch ▶ 0:41-1:11

2.1 $\text{lr} \cdot g / |g| = \text{lr} \cdot \text{sign}(g)$: every weight moves by exactly lr (0.01), up or down. Checked: with lr 0.05 the first step is 0.0500 for every weight.

2.2 10 and 1,000. $1 / (1 - 0.9)$ and $1 / (1 - 0.999)$.

3. Training the tiny GPT · rewatch ▶ 1:11-1:29

3.1 Adam (1.708) → SGD + momentum (1.940) → SGD lr 0.1 (2.476). From the episode's run.

4. Weight decay: Adam + L2 vs AdamW · rewatch ▶ 1:29-2:20

4.1 About 0.64. $(1 - 0.0003)^{1500} \approx 0.638$.

4.2 B. When $\text{wd} \cdot w$ dominates the gradient, Adam's normalization turns it into steps of about lr toward zero.

5. The cost, and the code · rewatch ▶ 2:20-2:38

5.1 56 GB. $2 \times 7 \times 10^9 \times 4$ bytes.

5.2 0.0500 for every weight (within 10^{-7}). The first step is $\text{lr} \cdot \text{sign}(g)$.